Ruido urbano en Bilbao y relación con el tráfico

Gorka De La Iglesia Perex

Introducción **2**
Descripción de los datos **2**
Metodología **2**
Sonómetro : Mediciones 3
Sonómetro: Ubicación 4
Unión de sonómetro mediciones y ubicación 5
Tráfico Bilbao 5
Unión de sonómetro con tráfico 8
Exportación dataset 9
Análisis exploratorio de datos 10
Gráfico 1 10
Gráfico 2 10
Gráfico 3 11
Gráfico 4 11
PowerBI **12**
Transformación de datos 12
Modelo en estrella 12
Creación de DAX 12
Análisis exploratorio de datos 13
Gráfico 1 13
Gráfico 2 14
Gráfico 3 14
Gráfico 4 15

Introducción

Este proyecto analiza datos reales de Bilbao para descubrir cómo influye el tráfico en el ruido que soportamos en las calles. Hemos cruzado la información del tráfico, la ubicación y los decibelios para identificar las zonas más ruidosas y sus horarios.

Descripción de los datos

Los datos utilizados en este estudio han sido extraídos en tiempo real del Portal Open Data del Ayuntamiento de Bilbao mediante su API pública.

El conjunto de datos procesado se estructura en dos tres principales:

- Sonómetros - Mediciones: Archivo JSON que recoge los valores de presión sonora (dBA) y la fecha/hora exacta de cada lectura.
- Sonómetros - Ubicaciones: Archivo GeoJSON estático que proporciona las coordenadas geográficas (latitud y longitud) para georreferenciar cada sensor.
- Aforos de Tráfico: Archivo GeoJSON que combina la geometría de los tramos de calle con la intensidad del flujo del tráfico.

Metodología

El flujo de trabajo se ha desarrollado íntegramente en lenguaje R, utilizando un proceso ETL (Extracción, Transformación y Carga) para preparar los datos antes de su visualización final en Microsoft Power BI.

Hemos comenzado importando las diferentes librerías.

```
# CARGA DE LIBRERÍAS
library(httr2)
library(jsonlite)
library(dplyr)
library(tidyr)
library(lubridate)
library(sf)
library(ggplot2)
```

Cambiamos la ruta para que la ejecución no de error.

```
# Directorio de trabajo
setwd("C:/Users/iabd/Desktop/RuidoBilbao")
getwd()
```

Sonómetro : Mediciones

Obtenemos los datos del enlace.

```
# URL del GeoJSON de sonometro_mediciones
url_sonometro_mediciones <-
"https://www.bilbao.eus/aytoonline/jsp/opendata/movilidad/od_sonometr
o_mediciones.jsp?idioma=c&formato=json"

# Leer JSON desde la web
sonometro_mediciones <- fromJSON(url_sonometro_mediciones)
```

Creamos un dataframe para los datos y le damos nombre a las columnas.

```
# Crear el dataframe de Sonometro_mediciones
sonometro_mediciones_df <- data.frame(
  Codigo = as.character(sonometro_mediciones$nombre_dispositivo),
  Decibelios = as.numeric(sonometro_mediciones$decibelios),
  FechaHora = as.POSIXct(sonometro_mediciones$fecha_medicion,
format="%Y-%m-%d %H:%M:%OS"),
  stringsAsFactors = FALSE
)
```

Comprobamos si hay nulos o duplicados.

```
# Revisar nulos y duplicadas
colSums(is.na(sonometro_mediciones_df))
sum(duplicated(sonometro_mediciones_df))
```

Creamos dos nuevas columnas: Hora y dia

```
# Extraer información adicional (Hora y Día)
sonometro_mediciones_df$Hora <-
format(sonometro_mediciones_df$FechaHora, "%H")
sonometro_mediciones_df$Dia <-
weekdays(sonometro_mediciones_df$FechaHora)
```

Realizamos diferentes comprobaciones para saber que está todo correcto.

```
# Mostrar ( Mínimo, Primer cuartil, Mediana, Promedio, Tercer
cuartil, Máximo )
summary(sonometro_mediciones_df$Decibelios)

# Mostrar la desviación Estándar
sd(sonometro_mediciones_df$Decibelios)

# Revisar  clases
class(sonometro_mediciones_df)

# Mostrar primeras lineas
head(sonometro_mediciones_df)
```

Sonómetro: Ubicación

Obtenemos los datos del enlace.

```
# URL del GeoJSON de sonometro_ubicacion
url_sonometro_ubicacion <-
"https://www.bilbao.eus/aytoonline/jsp/opendata/movilidad/od_sonometr
o_ubicacion.jsp?idioma=c&formato=geojson"

# Leer el GeoJSON como objeto sf
sonometro_ubicacion_sf <- st_read(url_sonometro_ubicacion, quiet =
TRUE)
```

Realizamos diferentes comprobaciones, si hay nulos o duplicados, para saber que está todo correcto.

```
# Revisar nulos en el dataframe
colSums(is.na(sonometro_ubicacion_sf))

# Revisar duplicadas en el dataframe
sum(duplicated(sonometro_ubicacion_sf))

# Revisar  clases
class(sonometro_ubicacion_sf)

# Mostrar primeras lineas
head(sonometro_ubicacion_sf)
```

Unión de sonómetro mediciones y ubicación

Realizamos la unión de ambos data frames y eliminamos los registros sin geometría.

```
# Asegúrate de que sonometro_ubicacion_sf tenga la columna geometry
activa
sonometro_sf <- sonometro_mediciones_df %>%
  inner_join(sonometro_ubicacion_sf, by = c("Codigo" = "name")) %>% #
inner_join para eliminar los que no tengan ubicación
  st_as_sf() # Esto funcionará SOLO si la columna 'geometry'
sobrevivió al join.

# Eliminar registros sin geometría válida
sonometro_sf <- sonometro_sf[!is.na(st_dimension(sonometro_sf)), ]

head(sonometro_sf)
```

Tráfico Bilbao

Obtenemos los datos del enlace.

```
# --- 1. CONFIGURACIÓN ---
url_trafico <-
"https://www.bilbao.eus/aytoonline/srvDatasetTrafico?formato=geojson"
ruta_trafico_local <- "traficoGEOJSON/trafico_final.geojson"
```

Desactivamos la geometría esférica estricta para evitar errores de cálculo en coordenadas planas.

```
sf_use_s2(FALSE) # Apagar geometría estricta
```

Descargamos los datos geoespaciales en tiempo real desde la web. Convertimos las columnas de texto a sus formatos correctos: fecha, hora y números. Seleccionamos solo las columnas útiles y la geometría, descartando el resto de información.

```
 --- 2. PASO 1: CREAR CON DATOS DEL ENLACE (LO NUEVO) ---
trafico_sf <- st_read(url_trafico, quiet = TRUE) %>%
  mutate(
    FechaHora = ymd_hms(FechaHora), # Aseguramos formato fecha
    Intensidad = as.numeric(Intensidad),
    Ocupacion = as.numeric(Ocupacion),
    Velocidad = as.numeric(Velocidad)
  ) %>%
  select(CodigoSeccion, Intensidad, Ocupacion, Velocidad, FechaHora,
geometry)
```

Añadimos el histórico al dataframe. Para ello, primero revisamos que existe el archivo verificando la ruta. Luego miramos que comparten la estructura y finalmente los unimos.

```
if (file.exists(ruta_trafico_local)) {
  # CAMBIO REALIZADO: Ahora usamos 'trafico_historico'
  trafico_historico <- st_read(ruta_trafico_local, quiet = TRUE)

  # TRUCO IMPORTANTE: El histórico guardado suele tener nombre
"FechaHora_trafico".
  # Para poder unirlo con el nuevo (que tiene "FechaHora"), hay que
renombrarlo temporalmente.
  if ("FechaHora_trafico" %in% names(trafico_historico)) {
    trafico_historico <- trafico_historico %>% rename(FechaHora =
FechaHora_trafico)
  }

  # Aseguramos que tengan las mismas columnas antes de pegar
  cols_comunes <- intersect(names(trafico_sf),
names(trafico_historico))
```

```
  # UNIÓN: Ponemos el nuevo arriba y el histórico debajo
  trafico_sf <- rbind(trafico_sf[, cols_comunes], trafico_historico[,
cols_comunes])
}
```

Comprobamos nulos y duplicados, hacemos la limpieza final y guardamos los datos en el fichero para tener más datos en la siguiente ejecución.

```
# Revisar nulos en el dataframe
colSums(is.na(trafico_sf))

# Revisar duplicadas en el dataframe
sum(duplicated(trafico_sf))

# --- 4. LIMPIEZA FINAL TRAS LA UNIÓN ---
# Eliminamos duplicados si se solapan fechas y renombramos para el
cruce posterior
trafico_sf <- trafico_sf %>%
  st_make_valid() %>%
  distinct(CodigoSeccion, FechaHora, .keep_all = TRUE) %>%
  rename(FechaHora_trafico = FechaHora)

# Revisar  clases
class(trafico_sf)

# Verificamos
head(trafico_sf)

# --- 5. GUARDAR  ---
st_write(trafico_sf, ruta_trafico_local, delete_dsn = TRUE, quiet =
TRUE)
```

Unión de sonómetro con tráfico

Preparamos los tramos de tráfico únicos (solo para calcular distancias).

```
# --- PASO 1: Encontrar la calle (Tramo) más cercana a cada sonómetro
---
trafico_tramos_unicos <- trafico_sf %>%
  distinct(CodigoSeccion, .keep_all = TRUE) %>%
  select(CodigoSeccion, geometry)
```

Buscamos el índice del tramo más cercano para cada sonómetro. Asignamos el ID de la calle (CodigoSeccion) al sonómetro

```
# Buscamos el índice del tramo más cercano para cada sonómetro
# Usamos 'sonometro_sf' que es tu variable correcta
idx <- st_nearest_feature(sonometro_sf, trafico_tramos_unicos)
# Asignamos el ID de la calle (CodigoSeccion) al sonómetro
sonometro_sf$CodigoSeccion <-
trafico_tramos_unicos$CodigoSeccion[idx]
```

Redondeamos la hora del SONÓMETRO y redondeamos la hora del TRÁFICO y preparamos la tabla para el cruce.

```
# --- PASO 2: Sincronización Temporal (Redondeo a 15 min) ---
# Redondeamos la hora del SONÓMETRO
sonometro_sf <- sonometro_sf %>%
  mutate(Clave_Tiempo = round_date(FechaHora, unit = "15 minutes"))

# Redondeamos la hora del TRÁFICO y preparamos la tabla para el cruce
# Usamos st_drop_geometry() para aligerar antes del join
trafico_sf_clean <- trafico_sf %>%
  st_drop_geometry() %>%
  mutate(Clave_Tiempo = round_date(FechaHora_trafico, unit = "15
minutes")) %>%
  distinct(CodigoSeccion, Clave_Tiempo, .keep_all = TRUE) # Evitar
duplicados
```

Unimos por: Mismo tramo de calle (CodigoSeccion) Y Mismo momento

```
# Unimos por: Mismo tramo de calle (CodigoSeccion) Y Mismo momento
(Clave_Tiempo)
dataset_final <- sonometro_sf %>%
  inner_join(trafico_sf_clean,
             by = c("CodigoSeccion" = "CodigoSeccion",
                    "Clave_Tiempo" = "Clave_Tiempo"))
```

Realizamos la limpieza y comprobamos que se han guardado los datos correctamente en el dataset_final.

```
# --- PASO 4: Limpieza y Validación ---
# (Opcional) Calculamos la diferencia real en minutos
dataset_final <- dataset_final %>%
  mutate(Dif_Minutos_Real = abs(as.numeric(difftime(FechaHora,
FechaHora_trafico, units = "mins"))))

# Eliminamos filas que no tengan datos de tráfico válidos (nulos)
dataset_final <- dataset_final %>%
  drop_na(Ocupacion, Intensidad, Velocidad)

sum(is.na(dataset_final))

# Verificamos el resultado
print(paste("Filas totales:", nrow(dataset_final)))
sum(is.na(dataset_final))
head(dataset_final)
```

Exportación dataset

Exportamos el dataset para su posterior uso en PowerBI

```
# Preparamos la tabla quitando la columna "geometry" que molesta en
el CSV
dataset_exportado <- dataset_final %>% st_drop_geometry()
sum(is.na(dataset_exportado))
# Guardamos el archivo CSV definitivo con el nombre que pediste
write.csv(dataset_exportado, "dataset_exportado.csv", row.names =
FALSE)
```

Análisis exploratorio de datos

Para poder identificar patrones de comportamiento, enriquecemos los datos generando nuevas variables a partir de la fecha y hora. Clasificamos cada registro según el Día de la Semana, el Tipo de Día y la Franja Horaria.

Gráfico 1

En este gráfico se compara la intensidad media del ruido a lo largo de las 24 horas, permitiendo visualizar las diferencias de comportamiento acústico entre los días laborables y los fines de semana.

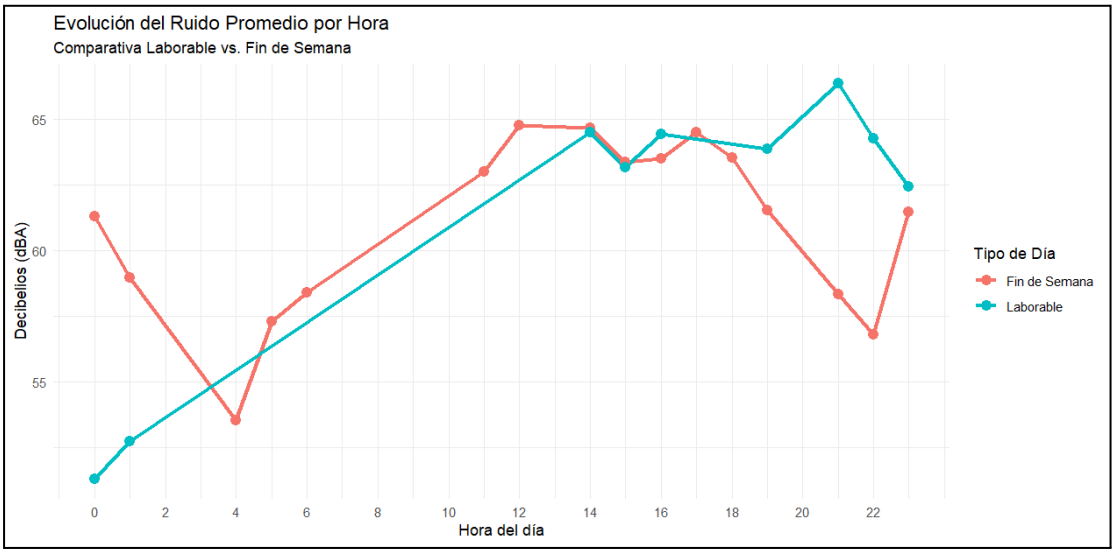


Gráfico 2

Top 10 puntos de medición que registran los niveles medios de decibelios más altos.

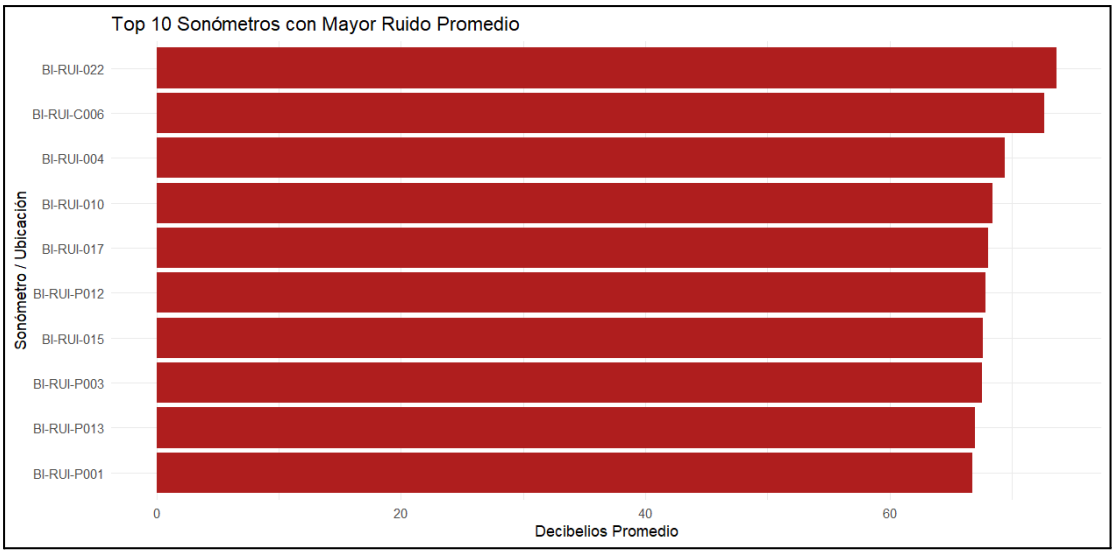


Gráfico 3

Matriz visual que cruza la hora del día con el día de la semana, utilizando una escala cromática para identificar rápidamente los patrones temporales de mayor intensidad acústica.

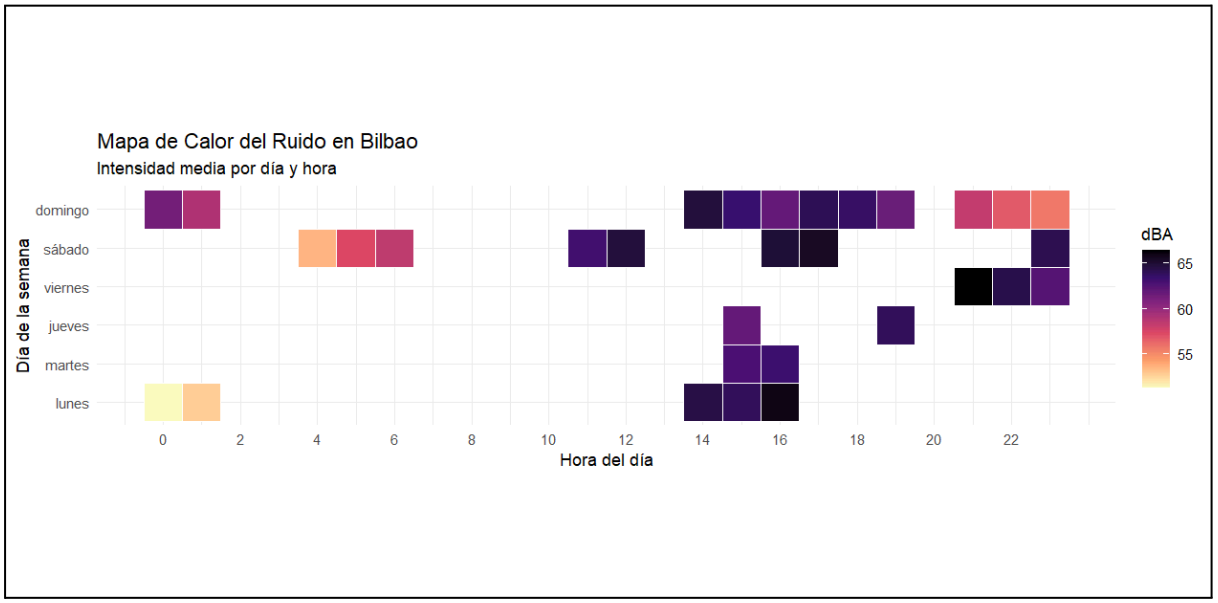
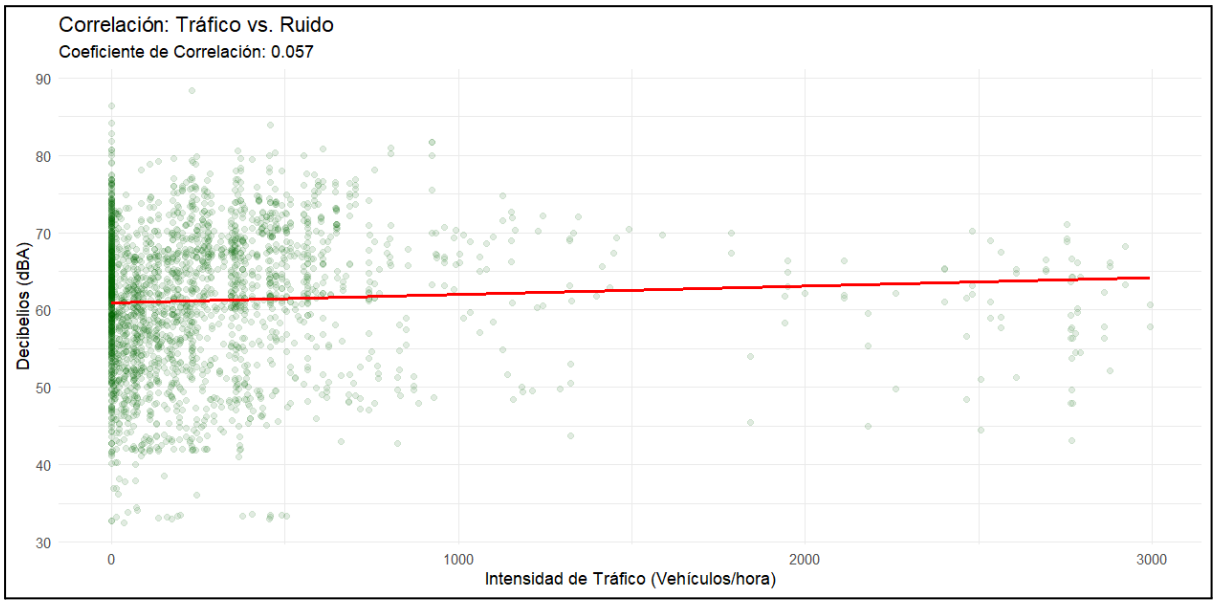


Gráfico 4

Diagrama de dispersión que relaciona la intensidad del tráfico con los niveles de ruido registrados, incluyendo una línea de tendencia para visualizar la correlación existente entre ambas variables.



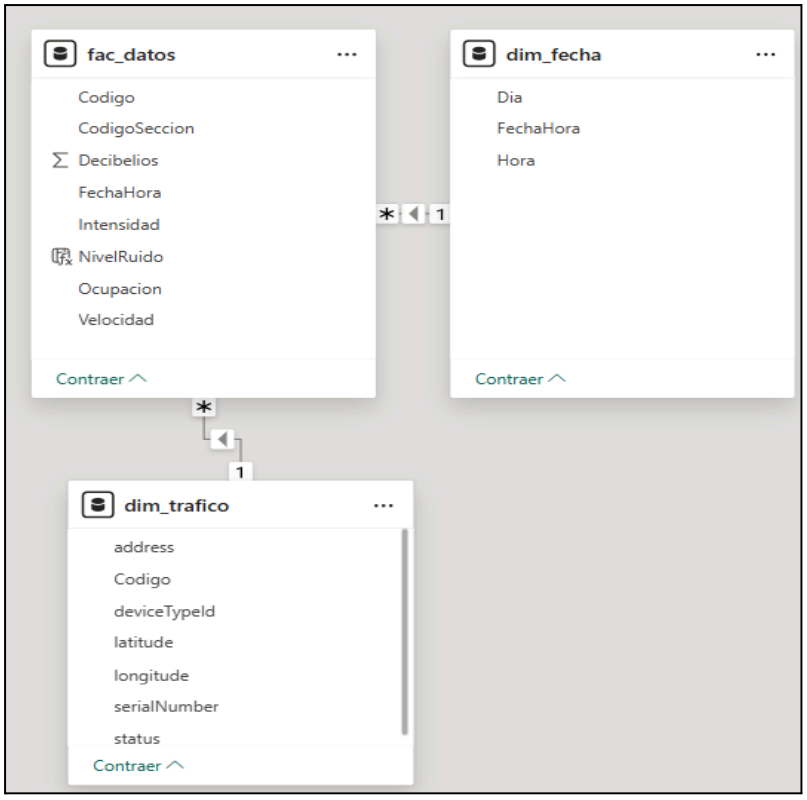
PowerBI

Transformación de datos

Para este paso utilizaremos el dataset exportado anteriormente. Lo primero que haremos será transformar las columnas de latitud, longitud y decibelios, para que salgan en su medida correcta.

Modelo en estrella

Una vez tenemos todos los datos correctamente, comenzaremos con la creación del modelo en estrella.



Creación de DAX

Con el modelo ya logrado crearemos un DAX, que nos permitirá saber si la media de los decibelios está en los parámetros permitidos.

```
Estado Ruido Medida =
IF(
```

```
    AVERAGE('Fact_Datos'[Decibelios]) > 65,
    "🔴 Fuera de Límite",
    "🟢 Nivel Seguro"
)
```

Análisis exploratorio de datos

Ahora si comenzaremos con el análisis con los diferentes gráficos

Gráfico 1

Promedio de decibelios por ubicación

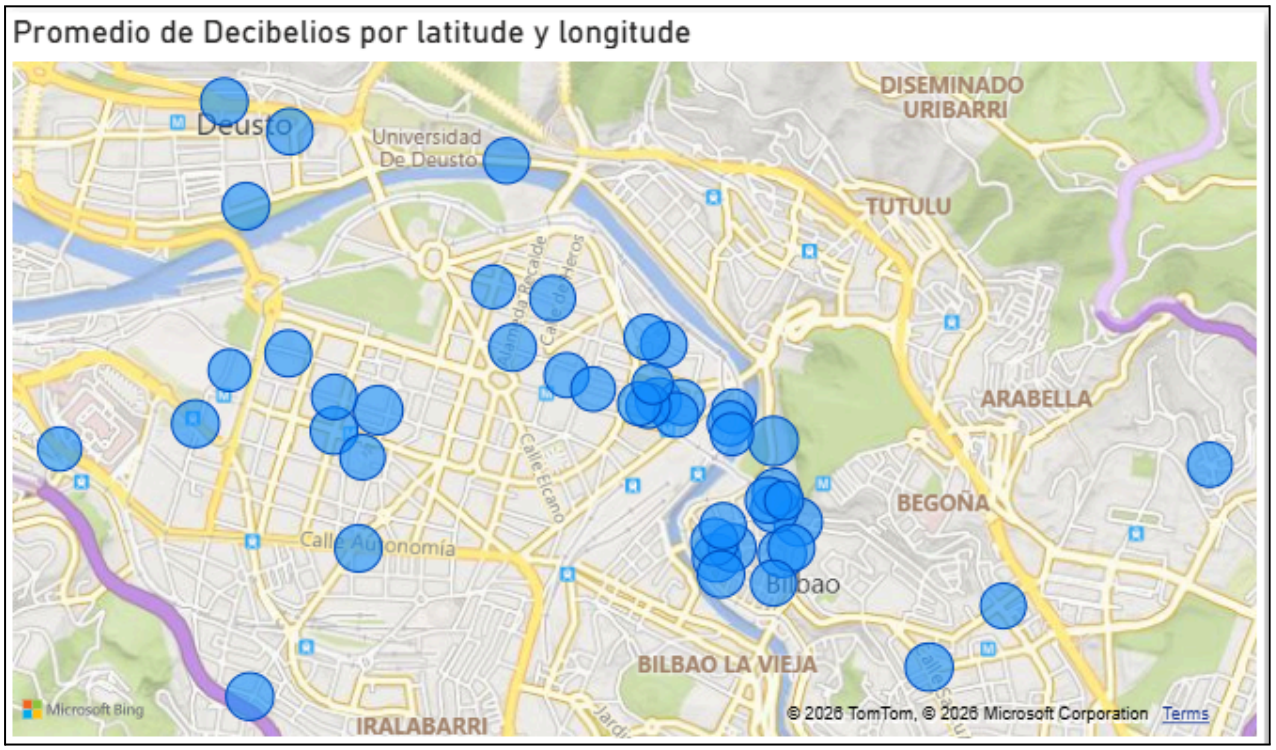


Gráfico 2

Promedio de decibelios por hora

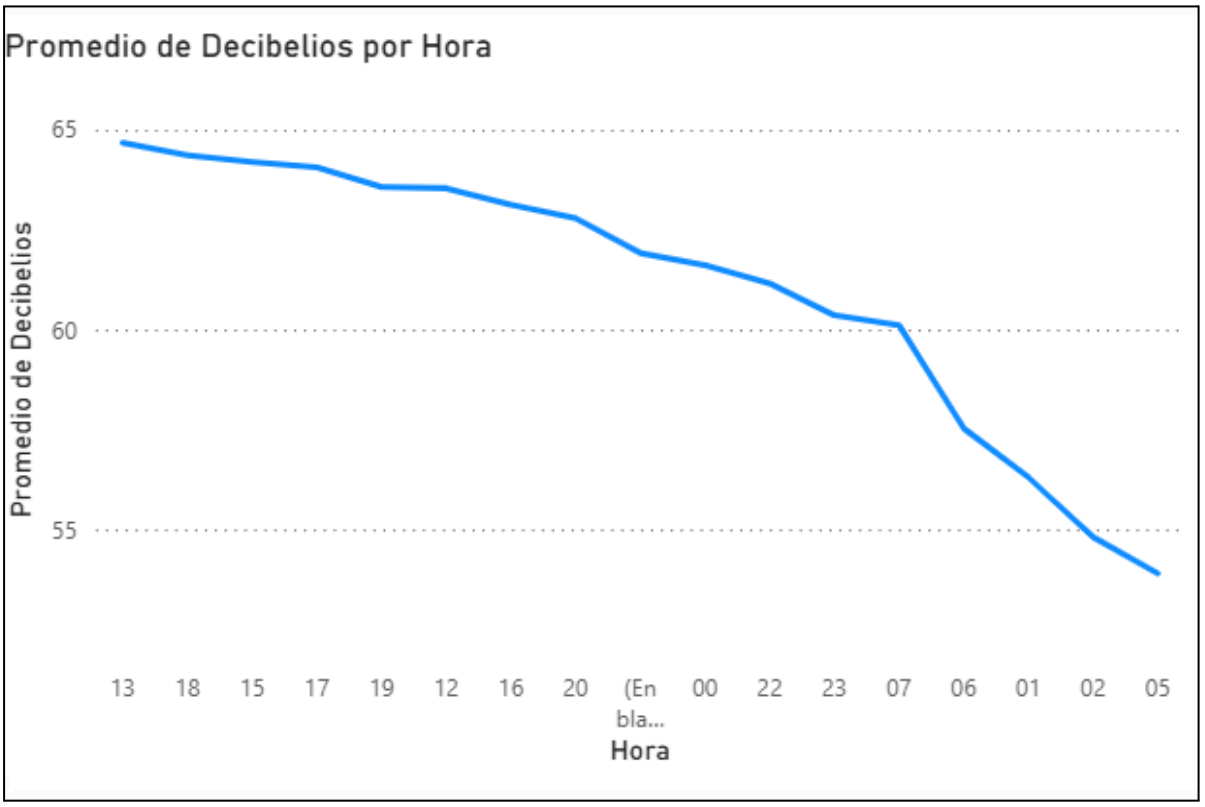


Gráfico 3

Promedio de decibelios por código

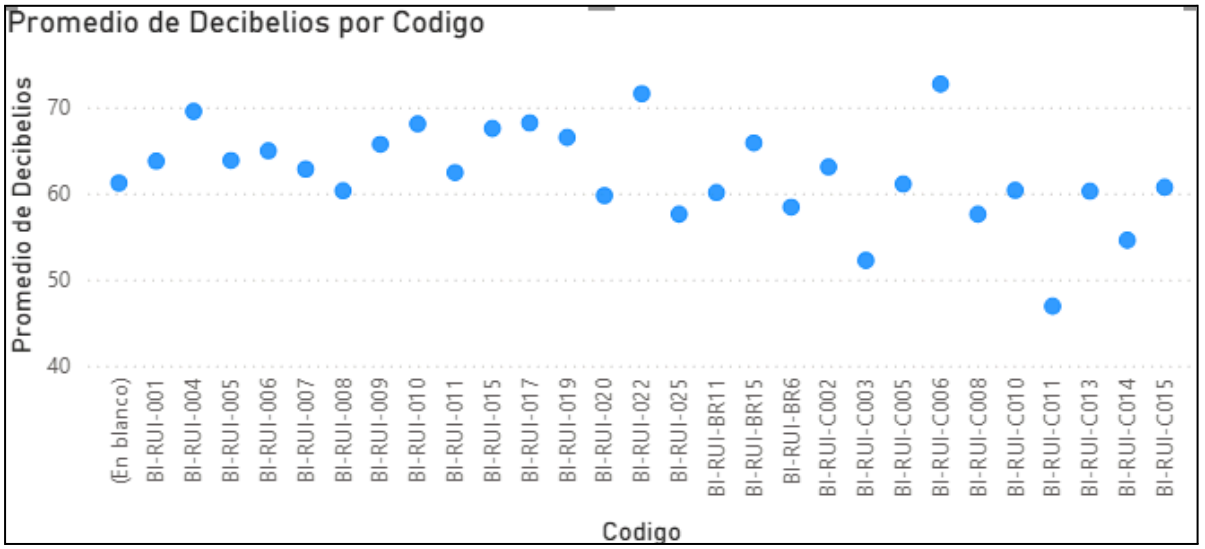
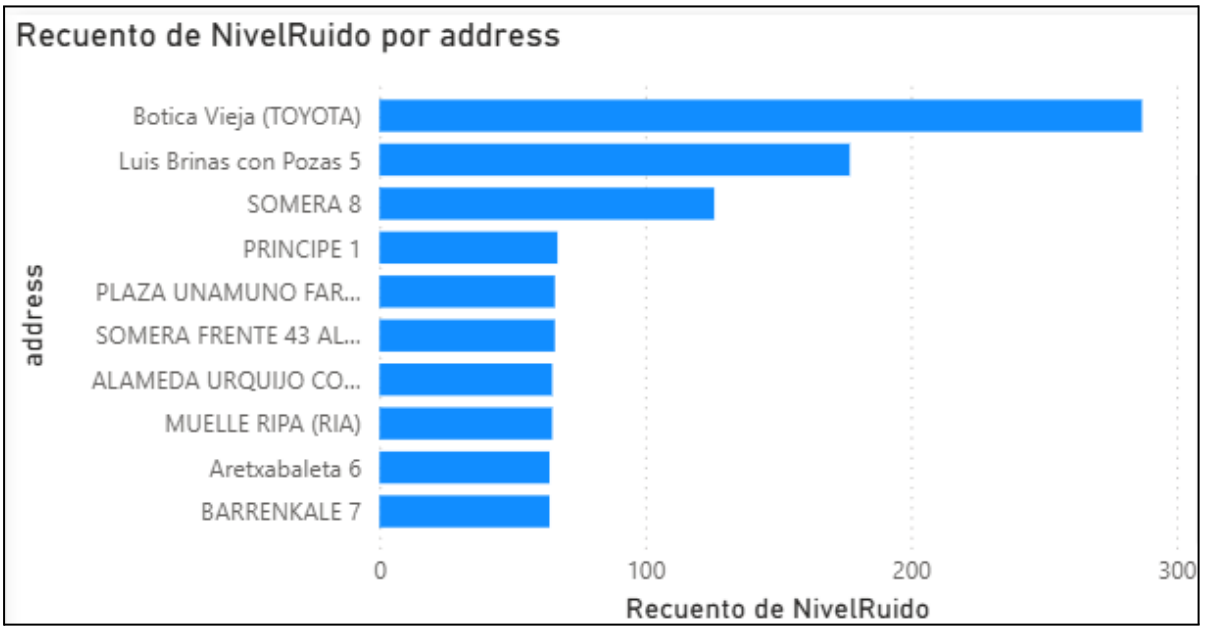


Gráfico 4

Recuento de nivel de ruido por dirección